

HDOC – DAY 4

C Programming

Multiplication of Two Integers Without Using the * Operator

1. Mistakes in the Given Algorithm

- It always adds num1 repeatedly, so the number of additions depends on num2. This is not the minimum possible number of additions.
- It assumes num2 is suitable as a positive loop count. A negative num2 can make the repetition condition fail immediately.
- It does not correctly handle negative × positive, positive × negative, and negative × negative cases.
- It does not explicitly handle multiplication by zero and therefore does not guarantee 0 additions for zero cases.
- It does not choose the smaller absolute value as the repetition count, so cases such as 3×12 use 12 additions instead of the required 3.

2. Corrected Algorithm

1. Define num1, num2, a, b, prod, count, sign.

2. Get num1, num2.

3. If num1 = 0 OR num2 = 0

- Set prod = 0
- Print prod
- Stop

4. Set sign = 1

5. If num < 0

- sign = -sign
- num1 = -num1

6. If num2 < 0

- sign = -sign
- num2 = -num2

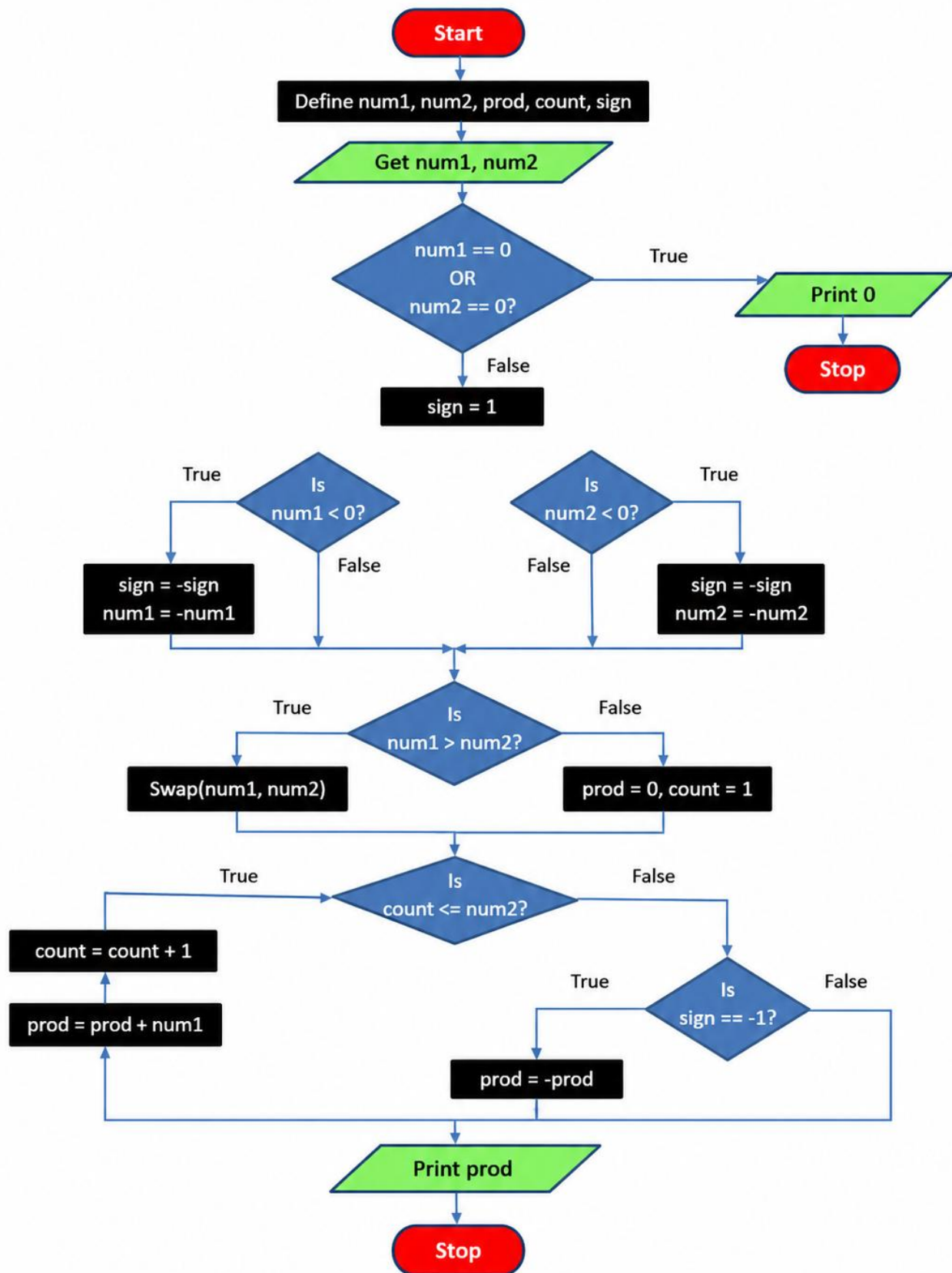
7. If num1 > num2

- Swap num1 and num2
8. Set prod = 0, count = 1
 9. Repeat until count > num1
 - prod = prod + num2
 - count = count + 1
 10. If sign = -1
 - prod = -prod
 11. print prod
 12. stop

3. Mistakes in the Given Flowchart

- The flowchart repeats the same incorrect logic as the algorithm: prod = prod + num1 is performed num2 times.
- There is no step for converting negative inputs to absolute values before repetition.
- There is no sign-handling decision for the four combinations of positive and negative integers.
- There is no proper zero check to guarantee 0 additions when either input is zero.
- The flowchart does not choose the smaller absolute value as the repetition count, so it fails the minimum-additions requirement.
- The loop condition is unsuitable for negative values of num2.

4. Corrected Flowchart



5. Verification Using the Given Test Cases

Eg. As in test case 1:

num1 = 12 and num2 = 3

since num1 > num2, swap them.

num1 = 3 & num 2 = 12

then,

prod = 0

prod = 0 + 12 = 12

prod = 12 + 12 = 24

prod = 24 + 12 = 36

Number of additions = 3

Output product = 36

Similarly the same is done to verify the rest of the cases;

Test	Input	Expected Output	Additions	Result
1	12 × 3	Product: 36	3	Pass
2	3 × 12	Product: 36	3	Pass
3	-6 × 5	Product: -30	5	Pass
4	6 × -5	Product: -30	5	Pass
5	-9 × -4	Product: 36	4	Pass
6	-4 × -9	Product: 36	4	Pass
7	0 × 8	Product: 0	0	Pass
8	8 × 0	Product: 0	0	Pass
9	0 × -7	Product: 0	0	Pass
10	-7 × 0	Product: 0	0	Pass
11	0 × 0	Product: 0	0	Pass
12	1 × -7	Product: -7	1	Pass
13	-1 × -9	Product: 9	1	Pass
